

False-Positive Classifier for Security-Scanner Findings

Model Summary & Evaluation Report

Pasupuleti Sai Manikanta | Energy Engineering, IIT Guwahati

Objective: Given a security-scanner finding on C source code, predict the probability that it is a **real vulnerability** (target = 1) rather than a **false positive** (target = 0), so that the costliest findings reach a human analyst first.

Headline Test-Set Results

0.956 TEST F2-SCORE	0.986 TEST ROC-AUC	0.987 TEST PR-AUC	0.972 RECALL (REAL VULN)	0.931 ACCURACY
------------------------	-----------------------	----------------------	-----------------------------	-------------------

Classification report

Class	Precision	Recall	F1	Support
False positive (0)	0.969	0.889	0.928	1353
Real vuln (1)	0.898	0.972	0.933	1353
Accuracy			0.931	2706

Confusion matrix (38 missed vulns)

	Pred: FP (0)	Pred: Vuln (1)
Actual FP (0)	1203	150
Actual Vuln (1)	38	1315

Cross-validation (5-fold, out-of-fold): ROC-AUC 0.977, PR-AUC 0.977. Operating threshold = 0.315 selected on CV only, never on the test set.

Metric Optimized & Rationale

For this project, I primarily optimized for the **F2-score**, while also monitoring **ROC-AUC** and **PR-AUC**. I chose F2-score because this is a vulnerability-detection problem, where missing a real vulnerability (a false negative) is significantly more costly than raising a false alarm (a false positive). Unlike the F1-score, which balances precision and recall equally, F2 weights recall four times more heavily, making it better suited to cybersecurity tasks where identifying as many real vulnerabilities as possible is the priority. To further align with this objective, I tuned the classification threshold using cross-validation and selected a value of **0.315**, which delivered strong recall (0.972 on the test set) while maintaining good precision.

Features That Mattered Most

The most important features came from the source code itself, represented using **TF-IDF features at both the word and character levels**. Word-level features captured meaningful programming constructs such as operators, keywords, and syntax patterns (for example `->`, `buf`, `casts`, and `chained checks`), while character n-grams captured lower-level structural patterns in the code. During exploratory analysis I identified **severe data leakage** in the metadata columns `path`, `repo`, `cwe_id`, `url`, and `sha`, each of which was perfectly correlated with the target label. To ensure the model learned genuine vulnerability patterns rather than memorizing shortcuts, I excluded these columns and trained on the `code` column only. I also removed overlapping code snippets between the train and test sets, making the evaluation more realistic and reducing the risk of inflated performance from memorization.

What I Would Do Differently With More Time

With more time, I would experiment with **transformer-based code models such as CodeBERT**, or with ensemble methods, to better capture the semantic patterns in source code. I would also perform deeper feature engineering and **robustness testing across different repositories and vulnerability types** — in particular a grouped-by-repository split — to confirm the model generalizes beyond the specific codebases seen during training and to further improve generalization.